

Resume-to-Job Automation Blueprint

Search · Score · Tailor · Track — never auto-apply

Prepared for Suryateja Orumpati · Senior AI Automation Architect

May 2026

Contents

1. Simple Meaning	1
2. What Will Be Automated vs Manual	2
3. End-to-End Workflow	2
4. Recommended Tools	3
5. ATS Scoring Logic	5
6. Resume Tailoring Logic	6
7. Local Folder Structure	7
8. Spreadsheet Schema	7
9. Dashboard Design	8
10. Step-by-Step Implementation Plan	9
11. Technical Architecture	9
12. Sample Pseudo-Code	10
13. Risks and Safeguards	11
14. Best Practical Version to Build First	11

Author: Senior AI Automation Architect & Career Workflow Designer (for Suryateja Orumpati)
Date: May 2026 **Target user:** New-grad SWE candidate, H1B sponsorship required, busy MS student
Design rule that governs everything below: the system **searches, scores, tailors, tracks, and stores** — it **never auto-applies**. Every application is opened and submitted manually by you.

1. Simple Meaning

In one sentence: **a local “co-pilot” that finds jobs, tells you how well your resume fits each one, rewrites your resume to fit better, and keeps a spreadsheet of where you applied — but lets you press the buttons.**

Think of it as a personal assistant sitting next to you:

1. You hand it your resume once.
2. It goes and collects open new-grad SWE jobs (from sources that allow it).
3. For each job it says: *“You’re an 82% match. You’re missing ‘Kubernetes’ and ‘CI/CD’. Here’s a version of your resume that scores higher — review it.”*
4. It hands you the job link. **You** open it and apply.
5. You click a ✓ tick. It writes a row into your local Excel tracker and files the exact resume PDF you used.

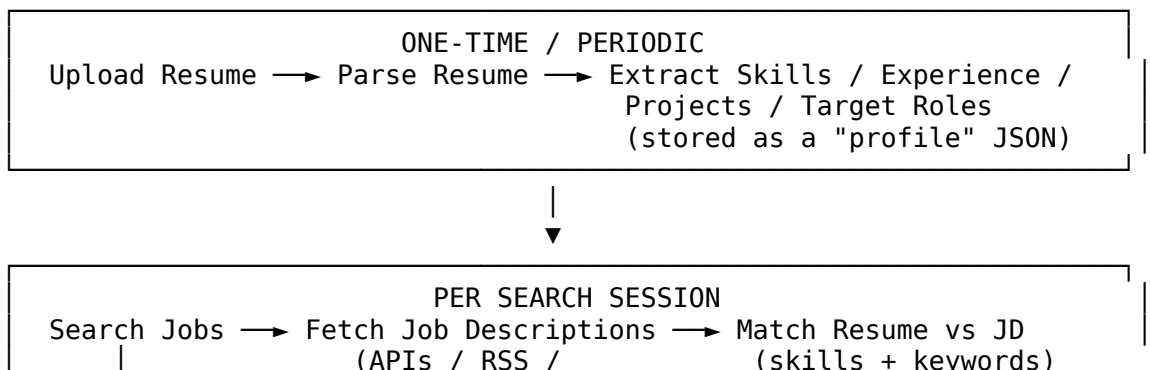
Nothing leaves your machine without you. No bot logs into job sites and clicks “Submit.” The automation removes the *busywork* (searching, comparing, formatting, record-keeping), not the *judgment* (deciding to apply, hitting submit).

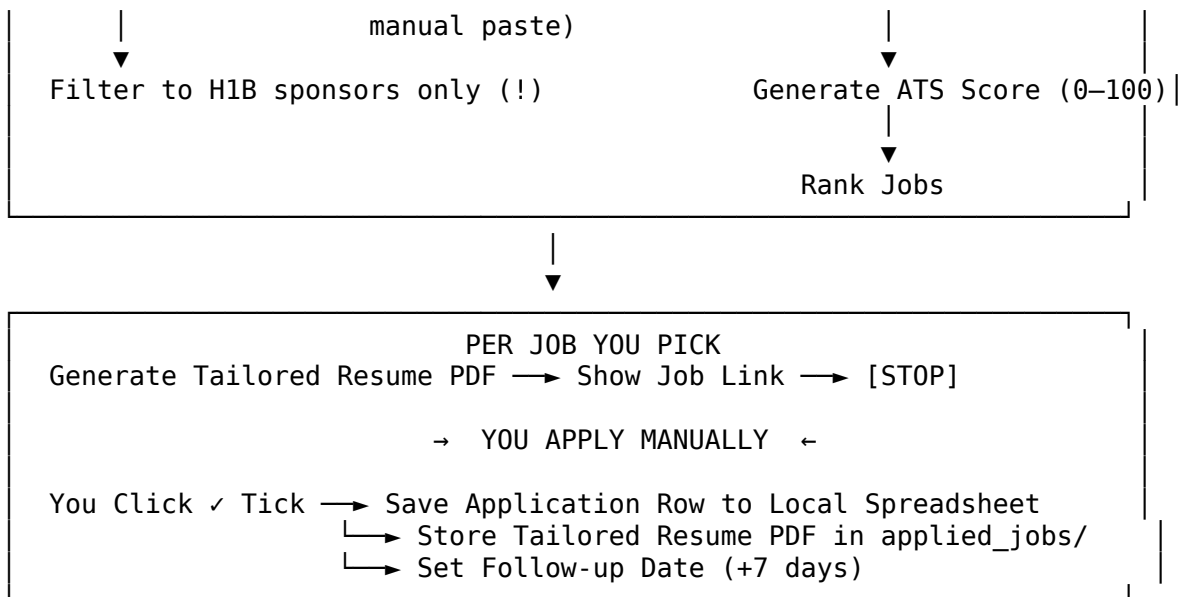
2. What Will Be Automated vs Manual

Step	Automated	Manual	Why
Parse resume into skills/experience/projects	✓		Deterministic text extraction
Search & collect job postings	✓		From APIs/RSS that permit it
Fetch job descriptions	✓		Only from allowed sources
Compute ATS match score	✓		Pure scoring math
Rank jobs by fit	✓		Sorting
Suggest resume improvements	✓		Keyword gap analysis + LLM
Generate tailored resume PDF	✓		Templating + PDF render
Review the tailored resume for accuracy		✓	You must verify nothing is exaggerated
Decide whether to apply		✓	Your call
Open job link & submit application		✓	(!) Never automated — ToS + safety
Click ✓ "Applied" tick		✓	Human confirmation gate
Write row to local spreadsheet	✓		Triggered by your tick
Save resume PDF to organized folder	✓		File management
Set follow-up reminders	✓		Scheduling

The golden line: automation stops at the job link. Crossing it (auto-filling and submitting applications) breaks site terms of service, risks bans, and can submit unreviewed content under your name. We never cross it.

3. End-to-End Workflow





Three loops, increasing in human involvement: the profile is built **once**, jobs are searched/scored in **batches**, and tailoring + applying happens **one job at a time** with you in the driver's seat.

4. Recommended Tools

4.1 Core stack (your preferred stack — validated)

Area	Tool	Notes
UI / Dashboard	Streamlit	Fastest path to a local web UI in pure Python; no JS needed
Language	Python 3.11+	One language for the whole system
Resume parsing	PyMuPDF (fitz) for PDF,	Fast, accurate text + layout extraction
Skill/keyword extraction	python-docx for Word	Rule-based first, LLM to fill gaps
Job matching	spaCy + a curated skills dictionary, optional LLM Keyword/TF-IDF scoring (scikit-learn) + optional LLM reasoning	Deterministic core, LLM for nuance
Spreadsheet	OpenPyXL	Native .xlsx read/write with styles & formulas
Data processing	pandas	In-memory tables, dedupe, ranking
Local DB (optional)	SQLite	Add when Excel gets slow (>1k rows)
PDF generation	ReportLab (programmatic) or Markdown → WeasyPrint (HTML/CSS → PDF)	WeasyPrint gives prettier, ATS-safe layouts
File storage	Local folders	See §7
Automation/orchestration	Python scripts + a scheduler (cron / Task Scheduler / your Cowork scheduled task)	Run searches on a cadence
LLM (optional)	Anthropic Claude or Google Gemini API	You already use Gemini in ClimateAI — reuse it

4.2 Job sources — the part most people get wrong

There is no single legal “search all jobs” API. The right answer is a **tiered source strategy**, safest first:

Tier	Source	Access	H1B relevance	Verdict
1	Greenhouse Job Board API	Public, no auth , not rate-limited	Stripe, Databricks, Snowflake & many top sponsors host here	★ Best
1	Lever Postings API	Public, no auth ; published postings may be consumed by third parties	Many mid-size sponsors	★ Best
2	Adzuna API	Free tier, needs app_id + app_key; aggregates boards + salary	Broad coverage, good for discovery	✓ Good
2	USAJOBS API	Free, API key; US federal jobs	(!) Federal jobs rarely sponsor H1B	– Niche
2	Jooble API	REST, request a key	Aggregator, decent coverage	✓ Good
3	RSS / XML feeds	e.g., Lever’s XML feed, some boards	Varies	✓ When offered
3	Manual URL paste	You paste a job link; system fetches that page	Universal fallback	✓ Always works
4	Browser-assisted	You drive the browser; tool reads the open page	For sites with no API	(!) Use sparingly
X	LinkedIn / Indeed scraping	ToS prohibits scraping; data behind login walls; Indeed’s public API restricted	—	X Do not

Endpoints (both public GET, no key):

Greenhouse: <https://boards-api.greenhouse.io/v1/boards/{company}/jobs>

Lever: <https://api.lever.co/v0/postings/{company}?mode=json>

The insight that makes this practical for you: a large share of H1B-sponsoring tech employers run their careers pages on **Greenhouse or Lever**, both of which give you a clean, free, no-auth JSON feed of every open role. So instead of “scraping the internet,” you keep a **list of ~40 target sponsor companies** and pull their boards directly. Reliable, free, and fully within terms.

You already have a search engine: your existing job-search skill uses WebSearch + your H1B fit rubric and outputs the exact JSON your command center expects. Keep using it for *discovery*; use Greenhouse/Lever APIs for *reliable structured pulls* from known targets. The new system consumes both.

5. ATS Scoring Logic

An ATS (Applicant Tracking System) ranks resumes mostly by **keyword and skill overlap** with the job description, plus parse-ability. We approximate that. **This score is an estimate to prioritize your effort — not the employer's real number.**

5.1 Scoring dimensions

#	Dimension	Weight	What it measures
1	Hard-skill match	30%	% of the JD's required skills present in your resume
2	Keyword coverage	25%	% of high-signal JD keywords (TF-IDF top terms) present
3	Title / role alignment	15%	Similarity of JD title to your target roles (SWE, Backend, Full-Stack)
4	Tools / technology match	10%	Overlap of named tools (Git, AWS, Postman, Docker...)
5	Experience / seniority fit	10%	Is it new-grad/0-2 yr? (penalize senior roles)
6	Domain match	5%	AI/ML, fintech, climate, etc. alignment with your background
7	Formatting / ATS-readiness	5%	Single column, standard headings, no images/tables that break parsers

Weights sum to **100**. Each dimension is scored **0-100**, then combined:

5.2 Formula

```
ATS_score =  
    0.30 * hard_skill_match  
    + 0.25 * keyword_coverage  
    + 0.15 * title_alignment  
    + 0.10 * tools_match  
    + 0.10 * experience_fit  
    + 0.05 * domain_match  
    + 0.05 * formatting_readiness
```

where each component $\in [0, 100]$, so $ATS_score \in [0, 100]$

Component definitions:

```
hard_skill_match = 100 * |resume_skills n jd_required_skills| / |jd_required_skills|  
keyword_coverage = 100 * |resume_keywords n jd_top_keywords| / |jd_top_keywords|  
tools_match      = 100 * |resume_tools n jd_tools| / |jd_tools|  
title_alignment  = cosine_similarity(jd_title_vec, target_roles_vec) * 100  
experience_fit   = 100 if role is 0-2 yr; 60 if 3 yr; 20 if senior/staff
```

domain_match = 100 if JD domain ∈ your domains else 50
 formatting_ready = rule checklist score (single column, std headings, text-based)

5.3 Bands and the missing-keywords report

Score	Band	Action
85–100	● Strong	Apply now; light tailoring
70–84	● Good	Tailor, then apply
55–69	● Stretch	Apply only if you can close the gaps honestly
< 55	● Weak	Usually skip (unless H1B options are thin)

Alongside the number, the system **always outputs a missing_keywords list** — the JD skills/keywords absent from your resume. That list is the single most useful output: it tells you exactly what to add (if true) or learn.

6. Resume Tailoring Logic

Iron rules (non-negotiable):

1. **Never fabricate** experience, employers, dates, or metrics.
2. **Never add a skill you don't have.** A missing keyword you genuinely don't know stays missing — the system *flags it as a gap to learn*, it does not insert it.
3. **Reorder and surface** what's relevant — move matching skills/projects up.
4. **Rephrase honestly** to mirror the JD's vocabulary (e.g., your "REST APIs" → keep, but if the JD says "RESTful microservices" and that's accurate, you may say so).
5. **Tailor three zones only:** the summary line, the skills ordering, and project/experience bullet emphasis.
6. **Keep it ATS-friendly:** single column, standard section headings, plain text, no text-in-images.
7. **Save each version as its own PDF**, named per job.

6.1 What gets tailored

Resume zone	Tailoring action	Example
Summary	Inject 2–3 true, JD-relevant keywords	"Backend-focused SWE (Python, Java, REST APIs) — built a live full-stack NLP app integrating 4 REST APIs."
Skills	Reorder so JD-matching skills appear first	If JD stresses Java + SQL, lead with those
ClimateAI bullet	Re-emphasize the facet the JD values	API-heavy JD → stress "4 integrated REST APIs, OAuth, JSON"; data JD → stress "Chart.js dashboard, CSV/PDF export"
Weed CNN bullet	Surface if JD is AI/ML; keep brief otherwise	ML role → lead with "97% accuracy, beat DeepWeeds benchmark"
Keywords	Add only true missing terms	JD wants "unit testing" and you do it → add; wants "Kubernetes" you don't know → flag, don't add

Per your CLAUDE.md: **ClimateAI and the Weed CNN always stay**; we swap *bullet emphasis* to match JD keywords, never the underlying facts.

6.2 Two-tier tailoring engine

- **Tier A — deterministic (no API key):** reorder skills, pick the best 2 of N pre-written bullet variants per project, inject true keywords into the summary from a controlled whitelist. Fully offline, instant, free.
- **Tier B — LLM polish (optional):** send the JD + your *real* bullets to Claude/Gemini with a strict prompt: *“Rephrase only; do not invent skills, employers, dates, or numbers. Mirror JD vocabulary where truthful.”* Then a **validator** diffs the output against your source facts and rejects any new company/skill/number that wasn’t in the original.

The validator is what keeps the LLM honest — tailoring can only ever **rephrase and reorder real content**.

7. Local Folder Structure

```
Job_Automation/
├── input_resume/           # your master resume(s): resume_master.pdf / .docx
├── job_descriptions/       # raw fetched JDs, one JSON per job (jd_<id>.json)
├── tailored_resumes/       # generated, not-yet-applied resume PDFs
├── applied_jobs/           # resume PDF actually used, filed after you tick ✓
│   └── <Company>_<Role>_<date>/resume.pdf
├── tracker/
│   └── job_applications.xlsx # the master spreadsheet (§8)
├── app/                   # the Streamlit code (app.py + modules)
├── config/
│   ├── profile.json        # parsed resume profile (skills/exp/projects)
│   ├── target_companies.txt # ~40 H1B sponsors w/ Greenhouse/Lever tokens
│   └── settings.yaml        # weights, paths, optional API keys
├── sample_data/           # example JD + example output to test with
└── logs/                  # run logs, search history, errors
```

Naming conventions: tailored PDFs are Resume_<Company>_<Role>_<YYYY-MM-DD>.pdf; applied jobs get their own dated subfolder so the exact artifact you submitted is preserved forever.

8. Spreadsheet Schema

File: tracker/job_applications.xlsx, sheet Applications.

Column	Type	Example	Notes
app_id	string	APP-20260529-001	Primary key, auto-generated
date_added	date	2026-05-29	When the job entered the system
company	string	Stripe	
job_title	string	Software Engineer, New Grad	
location	string	Remote / SF, CA	
job_link	url	https://...	Direct listing, not a search page

Column	Type	Example	Notes
source	string	Greenhouse / Lever / Adzuna / Manual	Provenance
ats_score	int	82	From \$5
hlb_sponsor	bool	TRUE	(!) Only TRUE if confident
resume_file	string	Resume_Stripe_NewGrad_2026-05-29.pdf	Exact file used
applied	bool	TRUE	Set only when you tick ✓
applied_date	date	2026-05-29	Stamped on tick
status	enum	Saved / Applied / OA / Phone / Onsite / Offer / Rejected	Pipeline stage
follow_up_date	date	2026-06-05	Auto = applied_date + 7
notes	string	Referred by X; OA due Fri	Free text

Status enum gives you a real pipeline view; follow_up_date powers reminders (\$10, Phase 6). The sheet is the single source of truth and stays 100% local.

9. Dashboard Design

A single local **Streamlit** app, five tabs, matching your existing command-center mental model:

Tab	You can...	Backed by
1. Resume	Upload master resume; see parsed skills/experience/projects; confirm profile	resume_parser.py → config/profile.json
2. Find Jobs	Pull from Greenhouse/Lever targets, query Adzuna, or paste a job URL/description ; run your job-search skill	sources.py
3. Match & Score	See every job with its ATS score, band color, and missing-keyword chips ; sort/filter; open the job link	ats.py
4. Tailor & Apply	For a chosen job: view suggestions, generate the tailored PDF, download it , click “✓ I Applied”	tailor.py, pdf_gen.py
5. Tracker	See all applications, pipeline counts, follow-ups due this week, edit status/notes	tracker.py (reads/writes the xlsx)

Key UX details: - The **“✓ I Applied”** button is the *only* thing that writes an “applied” row — the human confirmation gate. - Each job card shows the **score, the gaps, and the link** together, so you decide fast. - The Tracker tab surfaces **“Follow-ups due”** at the top so nothing slips.

(You can keep using your swe_application_command_center.html as a lightweight viewer; this Streamlit app is the engine that fills the same data model and adds parsing, scoring, tailoring, and PDF generation.)

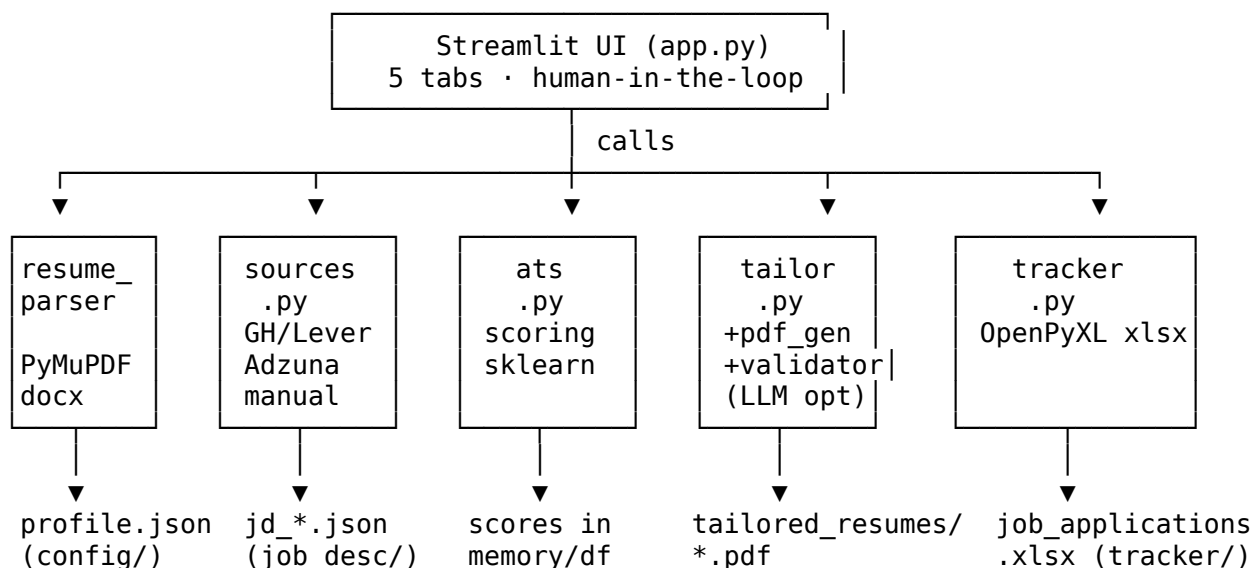
10. Step-by-Step Implementation Plan

Phase	Goal	Deliverable	Effort
1	Manual match	Paste 1 job URL/JD + read your resume → ATS score + missing keywords	½ day
2	Job search integration	Greenhouse/Lever pulls for your target list; Adzuna optional; plug in job-search skill	1-2 days
3	Tailored resume PDF	Tier-A tailoring + WeasyPrint/ReportLab PDF per job, with honesty validator	1-2 days
4	Local spreadsheet tracking	OpenPyXL write/append on ✓ tick; full schema (\$8)	½ day
5	Dashboard with tick	The 5-tab Streamlit UI wiring it all together	1-2 days
6	Scheduling & reminders	Daily search run + “follow-ups due” digest (cron / Task Scheduler / Cowork scheduled task)	½ day

Build in this order so you have something **useful after Phase 1** and a **complete loop after Phase 5**. Phase 6 is polish.

Recommended weekly cadence once built: schedule the search to run each morning, spend 20 min reviewing scored jobs over coffee, tailor + apply to your top 3-5, let the tracker handle the bookkeeping. That sustains your “10 apps/day” roadmap goal without the manual grind.

11. Technical Architecture



Stack options at a glance

Concern	Default (recommended)	Alternative
Language	Python 3.11	—
UI	Streamlit	Flask + HTML (you know Flask from ClimateAI)
PDF	WeasyPrint (HTML/CSS template)	ReportLab (programmatic)
Parse	PyMuPDF + python-docx	LLM extraction
Score	scikit-learn TF-IDF + sets	LLM scoring
Store	Excel (OpenPyXL)	SQLite when >1k rows
LLM	Optional (Gemini/Claude)	Fully offline (Tier-A only)

Everything runs **on your laptop**. The only outbound calls are to job-source APIs and (optionally) an LLM — both under your control via settings.yaml.

12. Sample Pseudo-Code

```
# — ONE-TIME: build profile —————
profile = parse_resume("input_resume/resume_master.pdf")
# → { skills:[...], tools:[...], projects:[...], target_roles:[...],
#     experience_years: 0, domains:["AI/ML","full-stack"] }
save_json(profile, "config/profile.json")

# — PER SEARCH SESSION —————
jobs = []
for company in load("config/target_companies.txt"):
    jobs += greenhouse_or_lever_pull(company)      # free, no-auth APIs
jobs += adzuna_search("new grad software engineer") # optional, keyed
jobs += job_search_skill_results()                # your existing skill
jobs += manual_pasted_jobs()                      # fallback

jobs = [j for j in jobs if is_new_grad(j) and is_hlb_sponsor(j.company)] # (!) filter

# — SCORE & RANK —————
for j in jobs:
    jd = fetch_description(j)                      # only from allowed sources
    j.ats, j.missing = ats_score(profile, jd)        # formula in $5
ranked = sort(jobs, by="ats", desc=True)

# — PER JOB YOU PICK (in the UI) —————
def on_generate(job):
    suggestions = suggest_improvements(profile, job.jd) # honest only
    tailored = tailor_resume(profile, job.jd)           # reorder+rephrase
    assert no_fabrication(tailored, profile)            # validator gate
    pdf = render_pdf(tailored,
        f"tailored_resumes/Resume_{job.company}_{job.role}_{today}.pdf")
    show(job.link, job.ats, job.missing, download=pdf)
    # [STOP] system STOPS here. No auto-apply.

def on_applied_tick(job, pdf):
    # fires ONLY when you click the tick
    move(pdf, f"applied_jobs/{job.company}_{job.role}_{today}/resume.pdf")
    tracker_append(
        # → job_applications.xlsx
        {
            "app_id": new_id(), "date_added": today, "company": job.company,
            "job_title": job.role, "location": job.location, "job_link": job.link,
            "source": job.source, "ats_score": job.ats, "hlb_sponsor": True,
            "resume_file": pdf.name, "applied": True, "applied_date": today,
            "status": "Applied", "follow_up_date": today + 7, "notes": ""
        }
    )

# — PHASE 6: scheduled —————
# every morning: run search + email/notify "N new matches, M follow-ups due"
```

13. Risks and Safeguards

Risk	Safeguard built into the design
Job sites block scraping / ToS violations	Use only APIs/RSS/manual paste that permit it; never scrape LinkedIn/Indeed ; keep a documented source per job (source column)
ATS score taken as gospel	UI labels it “ estimate, for prioritization ”; never shown as the employer’s real score
Tailored resume contains errors/exaggeration	Honesty validator rejects any new skill/employer/number; mandatory human review before download; you apply, not the bot
Auto-apply danger	There is no apply function . The system physically cannot submit. The ✓ tick only records <i>that you applied</i>
Sensitive data exposure	Everything is local ; API keys live in config/settings.yaml (git-ignored), never hard-coded; no resume data sent to any LLM unless you enable Tier B
Marking “applied” by accident	The tick is the only write path to applied=TRUE; it’s an explicit, deliberate click
Stale/expired job links	Store date_added; flag links older than N days; re-verify before applying
H1B false positives (!)	h1b_sponsor defaults to FALSE; set TRUE only via your verified sponsor list — never guess
LLM hallucination	Tier-A (offline) works with no LLM at all; Tier-B output is diffed against source facts before use

(!) **H1B-critical reminder:** the single most important filter is the sponsor check. Keep config/target_companies.txt as a *curated, verified* list (cross-checked against H1B disclosure data). A high ATS score at a non-sponsor is wasted effort.

14. Best Practical Version to Build First

Don’t build all six phases before using anything. **Build this minimal end-to-end slice first (Phases 1 + 3 + 4 + a thin UI):**

“Paste-and-Apply MVP” 1. Upload your master resume (parsed once). 2. **Paste one job description + its link.** 3. Get the **ATS score + missing-keyword list.** 4. Generate a **tailored resume PDF** (Tier-A, offline, honesty-validated). 5. Download it, apply manually, click ✓ **I Applied.** 6. Row lands in job_applications.xlsx; PDF filed in applied_jobs/.

This is **one screen**, needs **no API keys**, can’t break any site’s terms, and delivers the core value (score → tailor → track) on day one. Once it’s part of your routine, layer on Greenhouse/Lever auto-pull (Phase 2), the full 5-tab dashboard (Phase 5), and morning scheduling (Phase 6).

A runnable version of exactly this MVP ships alongside this blueprint — see app/ and the README. Run it, apply to a few real jobs this week, and grow it from there.

Reflection checklist (verified against this design)

- ✓ Job application stays **manual** — no apply function exists

- ✓ No unsafe auto-apply behavior
- ✓ Respects job-site rules (APIs/RSS/manual only; no LinkedIn/Indeed scraping)
- ✓ Local spreadsheet tracking (job_applications.xlsx, full schema)
- ✓ Unique tailored resumes saved locally per job
- ✓ ATS scoring logic with explicit formula + bands
- ✓ Realistic first version defined (Paste-and-Apply MVP)
- ✓ No fabricated resume content (iron rules + validator)
- ✓ Automation vs. manual clearly separated (the “golden line” at the job link)